

MDSimulationAgent Detailed Usage Guide

How to install, run, inspect, troubleshoot, and improve the agent-driven GROMACS workflow

What this repo is: an installable Python package named **mdagent** that wraps a GROMACS molecular-dynamics workflow in a CLI, a resumable artifact ledger, and three bundled Claude Code skills. It is strongest as a reproducible setup-and-validation assistant for soluble protein-only systems.

1. Package Map

The repository is organized around a small Python CLI plus bundled resources. The CLI lives in **src/mdagent/cli.py**, the ordered workflow lives in **src/mdagent/orchestrator.py**, one module per pipeline phase lives under **src/mdagent/steps/**, and the packaged schemas, tutorials, starter kit, and Claude skills live under **src/mdagent/_resources/**.

Area	Purpose
mdagent run-workflow	Runs ingest, classification, prep, topology, solvation, EM, NVT, NPT, production, analysis, optional visualization, and report.
mdagent prep-structure	Runs only ingest, classifier, and prep. It does not require GROMACS.
mdagent visualize	Writes VMD/PyMOL/NGL scripts and optionally renders checkpoint PNGs for an existing run.
mdagent install-skills	Copies the bundled Claude skills into ~/.claude/skills or a project .claude/skills directory.
mdagent init-project	Creates a runnable starter project with example configs, local skills, a bundled 1AKI PDB, and verify.sh.
mdagent pack-bundle	Builds a self-contained handoff folder or tarball, optionally with a wheelhouse for offline installs.
mdagent tutorials	Extracts or builds the packaged tutorial bundle.
mdagent doctor	Checks mdagent, GROMACS, RCSB connectivity, PROPKA, and viewer availability as needed.

2. Installation

For normal use, install the package as a uv tool, then install GROMACS for any step past structure prep. The README uses a placeholder GitHub path; replace it with the real remote or a local checkout path.

```
brew install uv
uv tool install --force git+https://github.com/<user>/MDSimulationAgent@v0.1.0
brew install gromacs
mdagent install-skills --user

mdagent --version
mdagent doctor --gmx-required
mdagent self-test resources
```

For local development in this repo:

```
uv sync
uv run pytest
uv run pytest --run-slow
uv run pytest --run-wheel
```

The base Python dependencies are jsonschema, pexpect, gemmi, and packaging. Optional extras include PROPKA for pKa-aware protonation and tutorial PDF generation dependencies.

3. Fastest Ways To Use It

A. Direct CLI run from any directory

```
mdagent run-workflow --runs-root ./runs --pdb-id 1AKI --run-id demo
mdagent inspect --run-root ./runs/demo
```

B. Stop at a partial workflow point

```
mdagent run-workflow --runs-root ./runs --pdb-id 1AKI --run-id prep-only --stop-after prep
mdagent run-workflow --runs-root ./runs --pdb-id 1AKI --run-id solvated --stop-after solvation
```

Valid stop points are **prep**, **topology**, **solvation**, **em**, **nvt**, and **npt**.

C. Use the starter kit

```
mdagent init-project ./my-md-project
cd ./my-md-project
./verify.sh
./verify.sh --run-smoke
mdagent inspect --run-root ./runs/smoke
```

D. Use Claude Code naturally

After the skills are installed, open Claude Code in a project and ask for a concrete MD task, such as *Set up lysozyme in water and minimize it* or *Prep this local PDB for simulation, but do not run dynamics*. The skills route those requests to the same CLI commands and surface structured failures from step reports.

E. Ship a bundle to someone else

```
mdagent pack-bundle ./mdagent-bundle --with-vendor --with-propka --archive
tar -xzf mdagent-bundle-macos-arm64-py311.tar.gz
cd mdagent-bundle
./setup.sh --check-only
./setup.sh
./run_simulation.sh
```

4. Run Configuration

Every run is controlled by a JSON object validated against `run_config.schema.json`. If you omit `--config`, the CLI materializes a minimal inline config from flags. For real work, copy one of the starter configs and edit it.

```
{
  "schema_version": "0.1.0",
  "pipeline_mode": "tutorial_reproduction",
  "interaction_mode": "noninteractive_defaults",
  "input": { "pdb_id": "1AKI" },
  "force_field": "oplsaa",
  "water_model": "spc",
  "box": { "geometry": "dodecahedron", "padding_nm": 1.0, "cutoff_nm": 1.0 },
  "ion_strategy": { "mode": "neutralize_only", "cation": "NA", "anion": "CL", "random_seed": 42 },
  "em": { "step_cap": 1000, "fmax_tol_kjmolnm": 1000.0 },
  "nvt": { "nsteps": 50000, "dt_ps": 0.002, "temperature_K": 300.0 },
  "npt": { "nsteps": 50000, "dt_ps": 0.002, "temperature_K": 300.0, "pressure_bar": 1.0 },
  "production": { "enabled": true, "nsteps": 500000, "dt_ps": 0.002 },
  "analysis": { "enabled": true }
}
```

Field	Meaning
pipeline_mode	tutorial_reproduction uses the GROMACS lysozyme tutorial style; general_md_prep switches default ingest to mmCIF and uses pdb2gmx -inter for titratable residue prompts.
interaction_mode	Schema exposes interactive, noninteractive_defaults, and strict_config_required, but the current automated topology path mostly uses planned/default answers.
input	Use either input.pdb_id for RCSB fetches or input.structure_path for local PDB/mmCIF files. Relative structure paths are resolved relative to the config file.
force_field / water_model	Passed to gmx pdb2gmx. Defaults are oplsaa and spc. The schema allows any force-field string, so local GROMACS availability still matters.
box	Controls editconf geometry and padding.
ion_strategy	neutralize_only, physiological_salt, or custom. The solvation step validates final charge accounting.
em	Short steepest-descent minimization, used as a validation gate.
nvt / npt / production	Length, timestep, temperature, pressure, and trajectory output stride.
visualization	disabled by default. Set mode/requested render options or run mdagent visualize later.
tool_versions	Optional pinned versions for reproducibility and fingerprinting.

5. Pipeline Internals

The orchestrator walks a fixed DAG. After each successful step, it writes a `step_report.json` and, for most computational steps, a `step_fingerprint.json`. The top-level `index.json` records status, attempts, artifacts, and fingerprint composites.

Step	What happens	Main outputs
00 preflight	Skipped as a step, but doctor preflight runs before compute.	doctor result surfaced on failure
01 ingest	Fetches RCSB PDB/mmCIF or copies local input; can derive PDB from mmCIF and verify coordinate ID injectivity.	original structure, working.pdb, optional coordinate_id_map.json
02 classifier	Counts residues and rejects unsupported chemistry.	classification.json
03 prep	Collects chain, HIS, CYS, and titratable-residue observations; strips HETATM in tutorial mode during ingest; optionally runs PROPKA if available.	observations.json, mutations.json, optional protonation_analysis.json

Step	What happens	Main outputs
04 topology	Drives gmx pdb2gmx through a PTY-based DialogueRunner, answering termini, disulfide, and optional per-residue prompts.	system_apo.gro/top, posre.itp, topology_plan.json, pdb2gmx transcript, protonation decisions
05 solvation	Runs editconf, solvate, grompp, genion, and a second grompp consistency gate.	system_ions.gro/top/tpr, charge_accounting.json
06 EM	Runs grompp and mdrun for steepest-descent minimization; parses em.log for convergence.	em.gro, em.log, em_convergence.json
07 NVT	Position-restrained constant-temperature equilibration.	nvt.gro/cpt/xtc/log/edr/tpr
08 NPT	Position-restrained constant-pressure equilibration from NVT checkpoint.	npt.gro/cpt/xtc/log/edr/tpr
09 production	Free production MD from NPT checkpoint; skipped when production.enabled is false.	production.gro/cpt/xtc/log/edr/tpr
10 analysis	Runs RMSD, Rg, RMSF, H-bonds, and energy summaries.	analysis.json and .xvg files
11 visualization	Optional static checkpoint rendering or script generation.	VMD/PyMOL scripts and optional PNGs
12 report	Regenerates REPORT.md from on-disk step reports and sidecars.	REPORT.md

6. Reading Outputs

The run directory is intentionally verbose. Treat it as a provenance bundle, not just a scratch folder.

```
runs/<run_id>/
  run_config.json
  index.json
  step_01_structure_ingest/working.pdb
  step_02_classifier/classification.json
  step_03_structure_prep/observations.json
  step_04_topology/system_apo.gro system_apo.top pdb2gmx_transcript.json
  step_05_solvation/system_ions.gro system_ions.top charge_accounting.json
  step_06_em/em.gro em.log em_convergence.json
  step_07_nvt/nvt.*
  step_08_npt/npt.*
  step_09_production/production.*
  step_10_analysis/analysis.json *.xvg
  REPORT.md
```

Readiness	Meaning
ready	Every gate passed, EM converged, and no chemistry/physics warnings were recorded.
ready_with_warnings	The run completed but chemistry or physics warnings deserve review.
blocked	A step failed or a blocking warning was emitted.
not_validated	EM did not run or needs a longer minimization, so the system should not be treated as validated.

For quick scientific sanity checks on lysozyme, inspect Rg around 1.4 nm, density near 1000 kg/m³ after NPT, and RMSD behavior. These are sanity checks, not proof of production-quality science.

7. Resume And Reproducibility

Re-run the same command with the same **--run-id** to resume. The orchestrator locks the run directory, recovers stale running steps, recomputes fingerprints for succeeded steps, invalidates stale descendants when configuration, inputs, schema, tool components, or source code changed, and continues from the first non-succeeded step.

```
mdagent run-workflow --runs-root ./runs --config ./run_configs/my_run.json --run-id demo
```

This makes the package useful for iterative setup: adjust a config, rerun, and keep the unaffected upstream artifacts.

8. Common Failure Triage

Failure	Where to look	Typical cause
UnsupportedResidueError	step_02_classifier/classification.json	Ligands, nucleic acids, modified residues, or HETATM records outside tutorial stripping.
CoordinateIdMapNotInjective	step_01_structure_ingest/coordinate_id_map.json	mmCIF to PDB derivation would collapse distinct canonical residues onto one PDB identifier.
UnexpectedPromptError	step_04_topology/pdb2gmx_transcript.json and step_report.json	GROMACS prompt changed or a prompt family is not recognized by the current catalog.
ConsistencyGateFailure	The failing step's step_report.json stderr tail	gmx grompp rejected topology, coordinates, mdp, or constraints.
ChargeAccountingMismatch	step_05_solvation/charge_accounting.json	Inserted ions do not match expected neutralization or final charge is not near zero.
EMDiverged / EMStuck	step_06_em/em.log and em_convergence.json	Bad contacts, poor starting geometry, topology issue, or too-strict/too-short EM settings.
No viewer detected	visualization/render_probe.json	VMD/PyMOL/NGLview unavailable; scripts are still written for later rendering.

9. Current Limitations

- Scientific scope is deliberately narrow: soluble protein-only systems are the supported path. Ligands, cofactors, nucleic acids, glycans, membranes, metal centers, covalent modifications, and mixed biomolecular assemblies fail fast or require manual preprocessing outside this package.
- The classifier is simple and mostly PDB-line based. It does not yet do rich biological assembly reasoning, membrane detection, OPM lookup, or full mmCIF chemistry classification.
- Structure prep is useful but still shallow. It observes HIS/CYS/titratable residues and can attempt PROPKA, but altloc resolution, MSE conversion, missing-loop repair, termini policy breadth, retained-water decisions, and general repair workflows are not mature.
- Topology automation depends on recognized pdb2gmx prompts. The recognizer was probed against GROMACS 2026.2 and may need prompt catalogs for other versions.
- PROPKA support is conditional. If the optional propka package is missing or fails, the code falls back to fixed pH-7 defaults. Current pKa-aware mapping is limited to the handled pdb2gmx prompt families and should be audited for unusual chemistry or non-OPLS force-field prompt orderings.
- The schema allows broad force-field strings, but the actual local GROMACS force-field directory, water model compatibility, and prompt order still govern success.
- Dynamics defaults are tutorial-sized. A 1 ns default production run, or the even shorter starter-kit smoke run, is not enough for publishable sampling.
- Execution is local only. The README explicitly notes that remote HPC/cloud/GPU execution is not wired yet.
- Analysis is a compact sanity suite, not a full scientific analysis workflow. H-bond analysis is best-effort, and statistical convergence is left to the user.
- Visualization is static checkpoint rendering, not trajectory movies or interactive trajectory analysis.
- Security and provenance are local-file oriented. The package writes rich artifacts, but it does not yet provide signed manifests for individual runs, formal audit logging across machines, or data-management integration.

10. Possible Improvements

Area	Improvement
Chemistry coverage	Add ligand/cofactor parametrization paths, nucleic-acid support, glycan handling, metal-center strategies, and explicit fail/ask policies for modified residues.
Structure prep	Implement robust altloc selection, MSE to MET conversion, missing-residue checks, disulfide planning, chain merge/split policies, retained-water classification, and better repair handoffs.
Prompt automation	Generate versioned prompt catalogs for multiple GROMACS releases and force fields; add discovery-mode fixtures that update recognizers safely.
Remote execution	Introduce a RemoteExecutor abstraction for Slurm/PBS/cloud runners, checkpoint sync, GPU queues, and resumable artifact transfer.
Configuration UX	Add a config wizard, schema-aware validation messages, force-field/water compatibility checks, and clearer profiles for common lab policies.
Scientific validation	Add longer equilibration recipes, replicate runs, convergence diagnostics, ensemble checks, automatic plots, and thresholds configurable by system class.
Analysis	Add plotting, PCA, clustering, secondary-structure analysis, binding-site metrics, trajectory stripping/centering, and export-ready report figures.
Visualization	Support movies, NGL web reports, trajectory snapshots over time, and richer default molecular representations.
Packaging	Replace placeholder GitHub URLs in docs, add release automation, publish wheels, and include platform-specific install checks.
Testing	Broaden golden paths beyond 1AKI, add more GROMACS versions, add real-world failure fixtures, and include regression tests for PROPKA-backed decisions.

11. Practical Guidance

- Use **prep-structure** first for unfamiliar structures; read classification.json before committing compute time.
- Keep each scientific hypothesis in its own config file and reuse run IDs only for intentional resume/update cycles.
- Pin GROMACS, mdagent, force field, water model, and random seeds when reproducibility matters.
- Read REPORT.md first, then step_report.json for any failed step, then the relevant sidecar JSON.
- Treat ready as a workflow validation result, not as a scientific conclusion.
- For real production, extend equilibration/production lengths and add domain-specific analysis outside the smoke-test defaults.